
Ensembling Tabular Q-Learning Agents for Competitive Pokemon

A Read-Only Inference Method with Heuristic-Prior Fallback

Arzaan Singh*

Department of Computer Science
Tulane University
New Orleans, LA
singharzaan@gmail.com

Abstract

[Sentence 1 (setup): single-agent tabular RL hits a ceiling effect in stochastic, partially-observable two-player games; Pokemon (gen4ou) is the benchmark.]

[Sentence 2 (method): we train K independent tabular $Q(\lambda)$ agents under identical hyperparameters but different random seeds and combine them at inference time using a read-only voting rule with a heuristic-prior fallback for unseen state-action pairs (no Q -table mutation at evaluation time).]

[Sentence 3 (headline): $K = 10$ ensemble reaches $66.8\% \pm 0.3\%$ win rate vs. `SimpleHeuristicsPlayer` over 100,000 battles, +12.6 pp over a compute-matched single-agent baseline trained for the same total simulated-battle budget.]

[Sentence 4 (analysis): the K -saturation curve has the Osband-style concave-up shape and plateaus by $K=10$; voting rule (soft / hard / confidence-weighted) has only a small effect at fixed K .]

[Sentence 5 (significance): first Q -function-ensemble study on Pokemon; the read-only inference rule is the load-bearing methodological contribution and is independent of the underlying value-function class.]

1 Introduction

Paragraph 1 — motivation. Tabular RL ceiling effect in stochastic, partially-observable two-player games. Pokemon as a 10^{12} -state benchmark. Why the single-agent ceiling is the interesting problem. One concrete sentence on Pokemon’s combination of move stochasticity, hidden opponent team, and turn-by-turn switch options. Cite Simões et al. [2020], Sahovic [2020] for the benchmark.

Paragraph 2 — why ensembling at all. Brief setup of the question: in supervised learning bagging variance-reduces; in value-based RL the picture is messier (Lin et al., 2024 shows shared-buffer deep ensembles can underperform their own single-agent baseline). For independent-buffer tabular agents, the question is open. Cite Wiering and van Hasselt [2008], Osband et al. [2016], Lan et al. [2020], Song et al. [2023].

Paragraph 3 — what is new in this paper specifically. List the three contributions cleanly: (a) a read-only inference rule for tabular Q -function ensembles with heuristic-prior fallback for unseen states (Equation 1 of Section 3), which keeps inference-time memory $O(K)$ in trained table size and avoids

*Research advisor: Dr. Zizhan Zheng, Department of Computer Science, Tulane University (zzheng3@tulane.edu). Prof. Zheng supervises this research line but is not an author of this paper.

the OOM failure mode of cached-update variants; (b) the first head-to-head comparison of soft / hard / confidence-weighted voting in tabular Q -learning at K up to 10 on a non-toy stochastic two-player game; (c) a compute-matched single-agent control trained on the same total simulated-battle budget (5×10^6 battles) as the ensemble ($10 \times 5 \times 10^5$ battles), so the +12.6 pp lift cannot be attributed to extra compute.

Paragraph 4 — result preview. State the headline numbers plainly. $K=10$ ensemble: $66.8\% \pm 0.3\%$ win rate vs. `SimpleHeuristicsPlayer` over 100,000 evaluation battles; +12.6 pp over compute-matched single agent; saturating K curve; voting-rule effect < 1 pp at fixed K .

Paragraph 5 — what this paper does NOT do. Inherited components (hierarchical action decomposition, heuristic Q -initialization) come from our prior factorial study [Singh and Zheng, 2026] and are treated as fixed inputs to V6, not contributions of this paper; full details in Appendix A. Deep-RL ensemble extensions are open questions and are not addressed here. Generalization across opponents and Pokemon pools is also out of scope and is enumerated as a limitation in Section 6.4.

2 Related Work

2.1 Ensemble methods in tabular reinforcement learning

One paragraph. Walk through the tabular ensemble line: Wiering and van Hasselt [2008] (first systematic study of voting rules in Q -learning), Faußer and Schwenker [2011] (ensemble methods with function approximation), Lan et al. [2020] (maxmin Q -learning as ensemble-style estimation-bias control), Peer et al. [2021] (ensemble bootstrapping for Q -learning). The distinction this paper draws: prior tabular ensemble work either lives in toy gridworld / small MDP regimes or uses algorithmic diversity (different update rules per member); we use seed-only diversity in a non-toy stochastic two-player game.

2.2 Ensemble methods in deep reinforcement learning

One paragraph. Cover the deep-RL ensemble line: Osband et al. [2016] (bootstrapped DQN, posterior-sampling exploration), Anschel et al. [2017] (averaged DQN for variance reduction), Chen et al. [2021] (randomized ensembled double Q -learning), Lee et al. [2021] (unified ensemble framework). Close with the Lin et al. [2024] “Curse of Diversity” failure mode: deep ensembles sharing a replay buffer can underperform their own single-agent baseline. Set up the contrast: our independent-table setup has no shared buffer and is shown to be immune by construction (full discussion in Section 6.3).

2.3 Reinforcement learning for Pokemon

One paragraph in roughly chronological order: Lee and Togelius [2017] (early Showdown AI competition), Kalose et al. [2018] (Stanford project, early Pokemon RL writeup), Sahovic [2020] (`poke-env`, the library used here), Simões et al. [2020] (deep RL on the Showdown simulator), Wang [2024] (MIT M.Eng. thesis on random-battle RL), Hu et al. [2024] (LLM-as-policy approach), Karten et al. [2025] (expert-level minimax LM agent), Grigsby et al. [2025] (transformer offline RL), Angliss et al. [2025] (VGC team-strategy benchmark). Close with the gap this paper addresses: to our knowledge, no prior paper trains K independent tabular Q -function agents under identical hyperparameters and combines them by inference-time voting on Pokemon battles.

3 Background and Method

3.1 Preliminaries

We model a Pokemon battle as a partially observable Markov decision process (POMDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma, \Omega)$. Here $\mathcal{S}$ is the set of joint game states, $\mathcal{A}$ is the player’s action set (move 1 through 4 plus a switch), P is the stochastic transition function implemented by the Pokemon Showdown simulator, r is a dense reward signal, γ is the discount factor, and $\Omega : \mathcal{S} \rightarrow \mathcal{O}$ is the observation function that hides opponent team information from the agent.

Each ensemble member is a single tabular agent trained with Watkins $Q(\lambda)$ [Watkins, 1989, Sutton and Barto, 2018]:

$$\delta_t = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \quad (1)$$

$$e_t(s, a) = \begin{cases} 1, & (s, a) = (s_t, a_t) \\ \gamma \lambda e_{t-1}(s, a), & a_t = \arg \max_{a'} Q(s_t, a') \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

$$Q(s, a) \leftarrow Q(s, a) + \alpha \delta_t e_t(s, a) \quad (3)$$

with replacing traces. Traces are cleared after any non-greedy exploratory action, preserving the off-policy correctness of Watkins’s algorithm.

Inherited components from prior work. Each ensemble member also uses two components developed in our prior factorial study [Singh and Zheng, 2026]: (i) *hierarchical action decomposition* of move-versus-switch into a master Q-table over a 20-tuple state and a switch sub-agent Q-table over a 17-tuple state; and (ii) *heuristic Q-value initialization* that seeds each new state-action pair with a small directional bias derived from type effectiveness, STAB, base power, and accuracy. We summarize both components in Appendix A; their design choices and the factorial study that selected the winning configuration are contained entirely in our prior work and are not contributions of this paper. For this paper, we treat the per-member training pipeline as a fixed black box and study what happens when K such agents are combined at evaluation time.

3.2 The ensemble inference rule

We now state the main methodological contribution.

Setup. At inference time we are given K trained tabular agents, each holding two Q-tables: a master Q_m^k and a switch sub-agent Q_s^k , for $k = 1, \dots, K$. We want to choose an action a^* at state s .

Read-only Q-matrix construction. We first enumerate the legal master actions $\mathcal{A}_m(s)$ and build the *Q-matrix* $\mathbf{Q} \in \mathbb{R}^{K \times |\mathcal{A}_m|}$. Each entry combines a member’s learned Q-value with a heuristic-prior fallback for state-action pairs the member has not observed:

$$\mathbf{Q}_{k,a} = \begin{cases} Q_m^k(s, a), & \text{if } (s, a) \in \text{dom}(Q_m^k) \\ h(s, a), & \text{otherwise,} \end{cases} \quad (4)$$

where h is the deterministic heuristic prior described in Appendix A. Crucially, Equation 4 *does not write* the heuristic value back into Q_m^k . In Appendix B we report a practical consequence: an earlier implementation that did cache the fallback into the dictionary grew memory at the rate of $O(\text{unique-states-encountered})$ and was killed by the operating system roughly 30,000 battles into a 100,000-battle evaluation; the read-only formulation keeps the memory footprint constant.

Combination rules. We compare three rules for collapsing the K rows of $\mathbf{Q}$ into a single chosen action.

Soft voting averages Q-values column-wise:

$$a_{\text{soft}}^* = \arg \max_a \frac{1}{K} \sum_{k=1}^K \mathbf{Q}_{k,a}. \quad (5)$$

Hard voting is plurality over per-member argmaxes, with ties broken by mean Q:

$$a_{\text{hard}}^* = \arg \max_a \left| \{k : a = \arg \max_{a'} \mathbf{Q}_{k,a'}\} \right|. \quad (6)$$

Confidence-weighted voting weights each member by the spread between its top Q-value and its row mean, then averages:

$$w_k = \max \left(\max_a \mathbf{Q}_{k,a} - \frac{1}{|\mathcal{A}_m|} \sum_a \mathbf{Q}_{k,a}, 0 \right), \quad (7)$$

$$a_{\text{conf}}^* = \arg \max_a \frac{\sum_k w_k \mathbf{Q}_{k,a}}{\sum_k w_k}. \quad (8)$$

If all $w_k = 0$ we fall back to soft voting.

Recursion into the sub-agent. If the chosen master action is the switch action, we apply the same combination rule to the sub-agent Q-tables $\{Q_s^k\}_{k=1}^K$ over the available switch candidates.

Soundness at $K = 1$. For a single member the three rules collapse to the same answer: the argmax of that member’s Q-row. This is a sanity check we use in Section 5.2 as the leftmost point of the K-saturation curve.

3.3 Training the ensemble

We train $K_{\text{max}} = 30$ members independently. Each member uses hyperparameters $\alpha = 0.1$, $\gamma = 0.99$, $\lambda = 0.7$ and a fixed exploration rate $\varepsilon = 0.05$. These values are fixed at the winning configuration of our prior factorial study [Singh and Zheng, 2026]; we do not re-tune them here. Member k is seeded with $\text{seed}_k = s_0 + k$; the only intentional source of diversity across members is the random sequence of exploratory actions each member draws.

All members are trained against SimpleHeuristicsPlayer from poke-env [Sahovic, 2020] in the gen4ou format, with teams drawn from a fixed 20-Pokemon pool through IndexedTeambuilder; the team given at battle index n is deterministic in n . Each member trains for 1,000,000 battles, for a total of 30,000,000 training battles. Training is parallelized across eight AWS EC2 c5.4xlarge cloud instances over fifteen days; full hyperparameter and reproducibility details are listed in Appendix C.

Evaluation budget. At evaluation time we cap the ensemble at $K = 10$ members. This cap is imposed by memory: each loaded member occupies approximately 5.5 GIB of resident memory in our pipeline, and at $K > 10$ we were unable to keep both the ensemble player and an opponent player resident on a single workstation. The remaining 20 trained members are held in reserve; the published evaluation in Section 5 uses the first 10 in seed order.

The training pipeline and the trained checkpoints are released at https://github.com/arzaansingh/poke_rl.

4 Experimental Setup

4.1 Environment

One short paragraph. State the stack: poke-env [Sahovic, 2020] on top of Pokemon Showdown, gen4ou format, fixed 20-Pokemon pool, IndexedTeambuilder deterministic team sampling (so members and opponents see identical matchups at the same battle index). Pool composition and the deterministic team-sampling recipe live in Appendix C; no need to repeat them here.

4.2 Evaluation conditions

Keep this list as-is in the final draft (numerics are locked). Voting strategies at $K=10$ (soft / hard / confidence); K -saturation sweep at soft voting ($K \in \{1, 3, 5, 10\}$); head-to-head ensemble vs. baseline_5m; baseline_5m solo vs. SimpleHeuristicsPlayer.

- **Voting strategies at $K = 10$:** SOFT, HARD, CONFIDENCE, each vs. SimpleHeuristicsPlayer.
- **K -saturation curve at SOFT:** $K \in \{1, 3, 5, 10\}$ vs. SimpleHeuristicsPlayer.

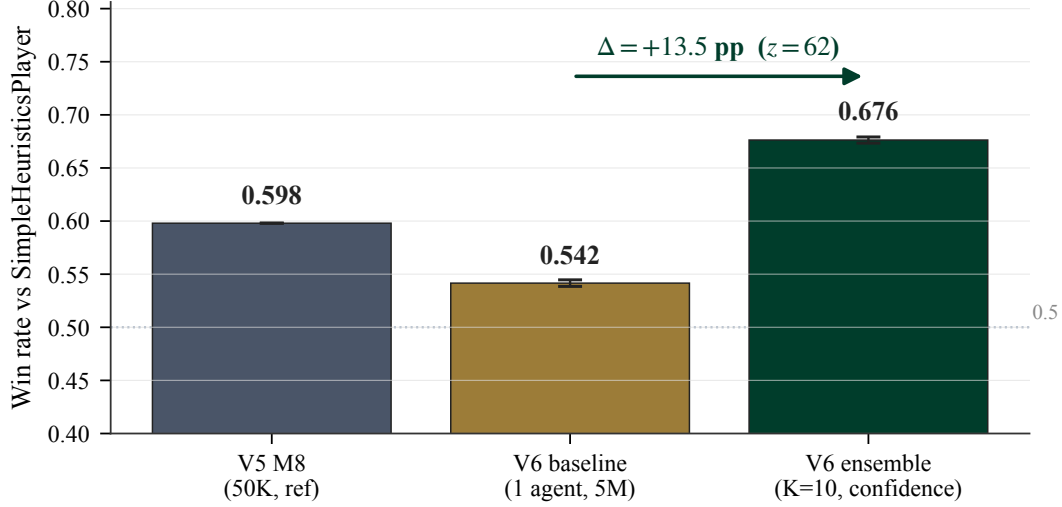


Figure 1: **V6 ensemble outperforms compute-matched single-agent baseline by 12.6 percentage points.** The $K=10$ ensemble (soft voting) achieves 66.8% win rate over 100,000 evaluation battles against SimpleHeuristicsPlayer, compared to 54.2% for baseline_5m, a single HierSmartPlayer trained for $5\times$ longer on identical hardware. Error bars show Wilson 95% confidence intervals.

- **Head-to-head:** $K=10$ SOFT ensemble vs. baseline_5m (compute-matched single agent).
- **Baseline solo:** baseline_5m solo vs. SimpleHeuristicsPlayer.

4.3 Evaluation protocol

Keep the numerics. $N = 100,000$ battles per condition, Wilson 95% intervals [Wilson, 1927], normal-approximation difference-of-proportions test for between-condition comparisons. Brief prose pass to integrate this into the paragraph after the meeting.

Each condition is evaluated over $N = 100,000$ battles. Win rates are reported with Wilson 95% confidence intervals [Wilson, 1927]. Differences in proportions use the normal approximation to the difference of independent binomials. All hardware specs and elapsed wall-clock times are tabulated in Section C.

4.4 Baseline opponents

One paragraph for each baseline. (i) SimpleHeuristicsPlayer: poke-env’s reference heuristic [Sahovic, 2020]; standard opponent in Wang [2024], Grigsby et al. [2025], Angliss et al. [2025]. (ii) baseline_5m: a single HierSmartPlayer trained for 5×10^6 battles ($5\times$ each ensemble member’s budget) under identical hyperparameters, hardware, and pool; the compute-matched single-agent control. Make this distinction explicit — without it the $+12.6$ pp lift in Section 5.1 could be attributed to extra wall-clock compute rather than to ensembling.

5 Results

5.1 Ensemble beats compute-matched single agent

One paragraph: state the headline numbers ($K=10$ soft = $66.8\% \pm 0.3\%$, baseline_5m = $54.2\% \pm 0.3\%$, $\Delta = +12.6$ pp, $z \approx 62$, $p \ll 10^{-3}$), then say what they mean. Key argument: the ensemble is compute-matched, so the lift is not buyable with more single-agent training; it is attributable to the ensemble construction itself. Forward-reference Section 6.1 for the mechanism.

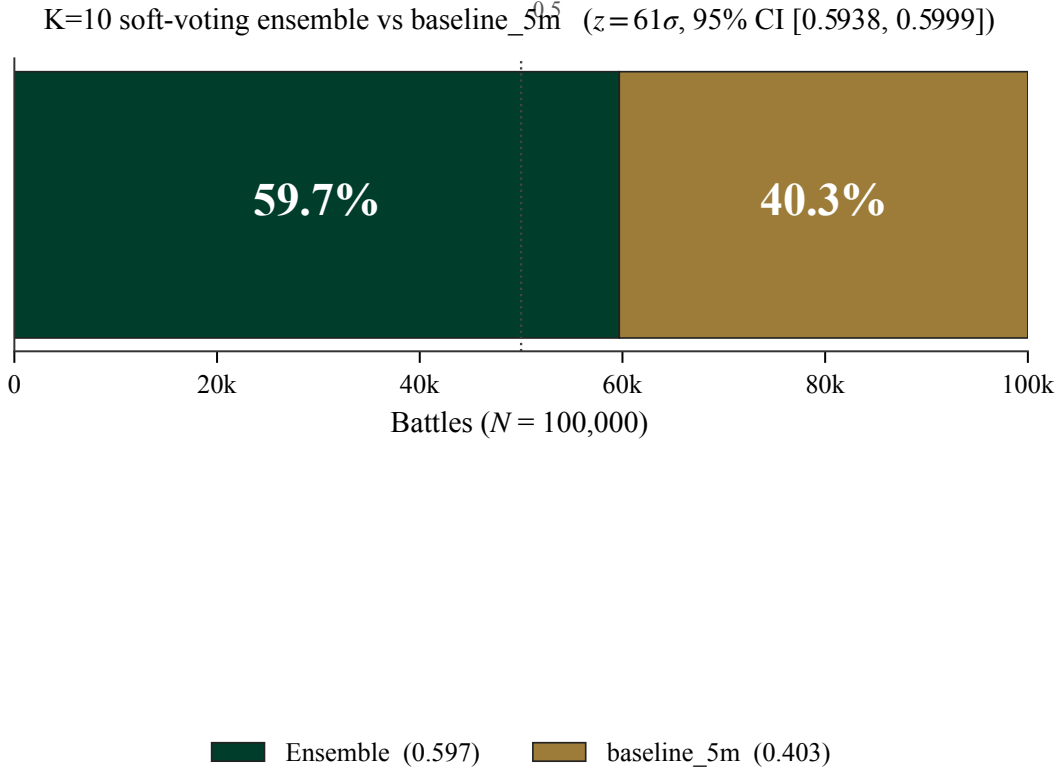


Figure 2: **Head-to-head: the $K=10$ ensemble wins 59.7% of matches against baseline_5m.** Each side picks teams from the same 20-Pokemon pool via the deterministic IndexedTeambuilder, so any non-coin-flip win rate is a methodology effect, not a team-matchup artifact.

One paragraph on Figure 2: when ensemble and baseline_5m face each other directly (same pool, same deterministic team sampler), the ensemble wins 59.7% of matches. This is stronger than the indirect-via-heuristic comparison because both agents face an opponent of identical strength on every battle, so the non-coin-flip outcome rules out the “the heuristic just happens to be weak against ensembles” alternative hypothesis.

5.2 K -saturation curve

One paragraph: report the curve ($K=1$: 55.9%; $K=3$: 62.7%; $K=5$: 65.1%; $K=10$: 66.8%) and note its concave-up saturating shape. Note that this matches Osband-2016’s diminishing-returns prediction for posterior-style ensembles. Be explicit that the $K=10$ cap is a workstation memory limit (members ≈ 5.5 GiB each), not a methodological choice; whether the curve plateaus permanently or has a second knee beyond $K=10$ is unresolved and is listed as a limitation in Section 6.4.

5.3 Voting strategy comparison

One paragraph: SOFT 66.8%, HARD 65.6%, CONFIDENCE 67.6%. All three within 2 pp; confidence-weighted is the nominal winner but the difference vs. soft is within the binomial CI overlap. Brief mechanism: HARD discards Q -value magnitude (vote becomes information-poor), so it loses some lift; CONFIDENCE preserves more information but the marginal gain is small at $K=10$ because soft voting already averages magnitudes. Take-away for the reader: rule choice matters less than the existence of an ensemble.

5.4 Per-member diagnostics

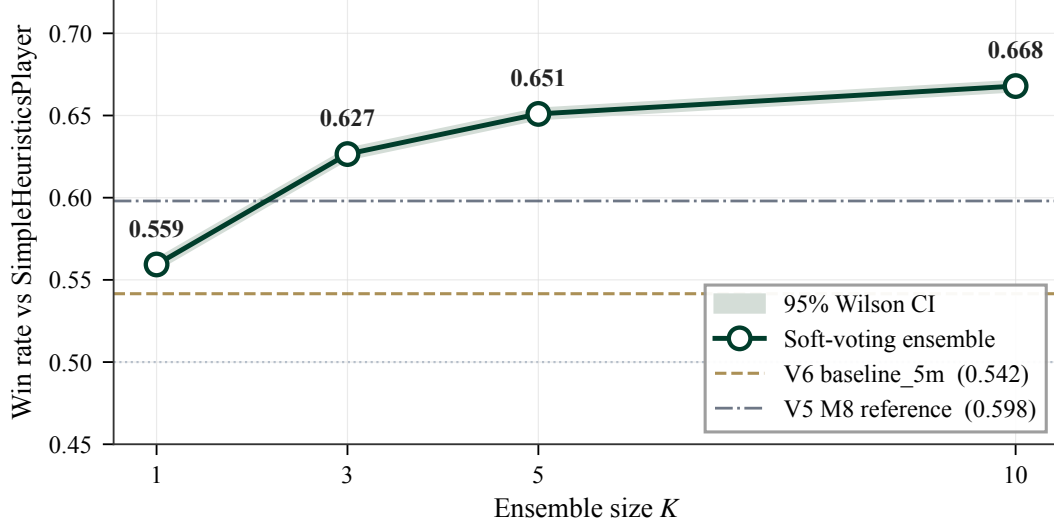


Figure 3: **Performance saturates near $K=10$ in the predicted Osband-2016 shape.** Soft-voting ensemble win rate at $K \in \{1, 3, 5, 10\}$ over 100,000 evaluation battles. Shaded region: Wilson 95% CI. The $K=10$ ensemble lifts win rate by 10.9 pp over the single-agent ($K=1$) baseline.

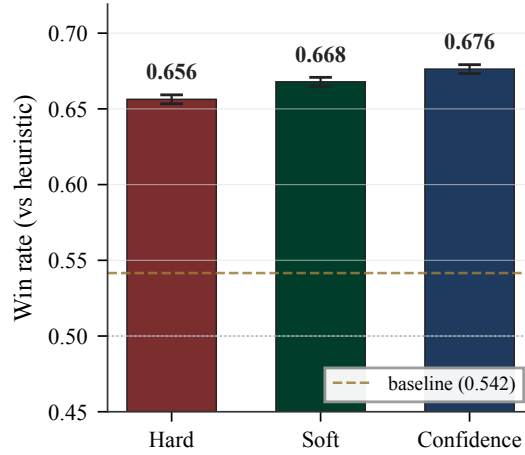


Figure 4: **Voting strategy choice has limited effect at $K=10$.** All three rules sit within 2 pp of each other. Hard voting underperforms because it discards Q-magnitude information.

Two short paragraphs. (i) Per-member solo distribution: mean solo win rate is in the weak-learner regime ($\sim 55\text{--}57\%$), so the ensemble is genuinely converting weak learners into a stronger predictor (Schapire framing). (ii) Disagreement + fallback diagnostics: non-zero pairwise argmax disagreement means the members have not all collapsed onto the same policy — the ensemble is doing real combination work. The fallback rate quantifies how often the inference rule actually invokes the heuristic prior; the modest observed rate is why the read-only rule does not visibly hurt performance.

5.5 Comparison to prior work

One paragraph situating V6 in the published literature. Important caveats to spell out so reviewers do not flag them: (i) Wang [2024] uses random battles, not gen4ou OU, so the comparison is qualitative; (ii) Grigsby et al. [2025] uses massive offline data and a transformer, so the comparison is about position in the landscape not method-vs-method; (iii) our number is reported with $N=10^5$ battles, a larger evaluation than most prior work, so the CI is tighter. Take-away: tabular ensembling

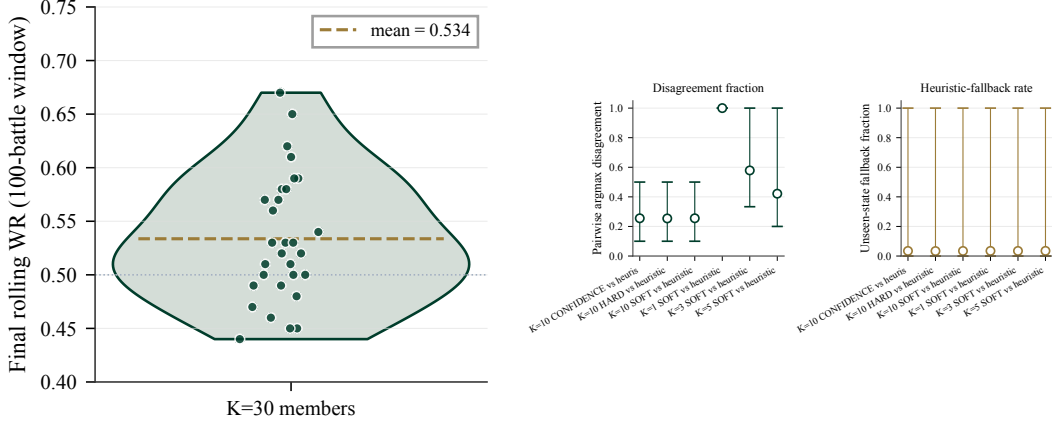


Figure 5: **Left:** distribution of final solo win rates across the 30 trained members (mean dashed). Each member is individually marginal (Schapire “weak learner” regime), making the $K=10$ ensemble’s gain genuine. **Right:** pairwise argmax disagreement (left panel) and unseen-state fallback rate (right panel) per evaluation condition. Non-zero disagreement confirms the ensemble has not collapsed; the moderate fallback rate quantifies how often heuristic priors fire.

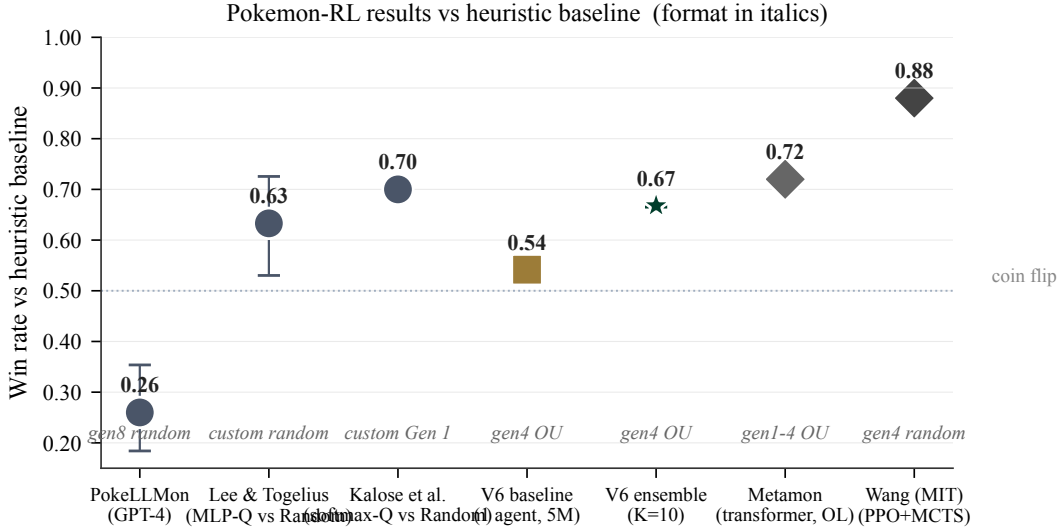


Figure 6: **V6 in context.** Published Pokemon-RL win rates against poke-env’s SimpleHeuristicsPlayer or an equivalent heuristic baseline. V6 ensemble at 66.8% in gen4ou lands within Metamon’s reported band for early-generation OU formats [Grigsby et al., 2025] at orders-of-magnitude less compute, well above all prior tabular work [Lee and Togelius, 2017, Kalose et al., 2018] and pre-search LLM agents [Hu et al., 2024].

sits competitively in the landscape while using orders-of-magnitude less compute than the deep-RL endpoints.

6 Discussion

6.1 Why the ensemble works in this regime

One-paragraph mechanism story. Each member’s Q -table is trained from a different stochastic trajectory through the same MDP, so per-state action-value estimates have errors that are largely uncorrelated across members. Voting averages those errors out; the expected vote argmax converges

toward the true argmax faster than any single member. Cross-reference the per-member disagreement diagnostic (Figure 5, right) to show the diversity assumption is satisfied in practice, not just in theory.

6.2 Why ensembling beats more single-agent compute

One-paragraph paragraph framing the compute-matched comparison. `baseline_5m` plateaus around 54% after 5×10^6 battles; ten agents at 5×10^5 battles each cover the state space more broadly than one agent at 5×10^6 does, even though total simulated battles are identical at 5×10^6 . Tie this to the K -saturation curve (Figure 3): the lift comes from state-space coverage breadth, not from extra wall-clock work. This is the “coverage-versus-depth” trade-off the paper makes explicit.

6.3 Avoiding the Curse of Diversity

The recently-documented “Curse of Diversity” failure mode for ensemble-based exploration [Lin et al., 2024] arises when ensemble members share a replay buffer: members trained on largely off-policy data relative to their own policy underperform a single-agent baseline. By construction, the ensemble in this paper does not share a buffer or any training data across members: each member is a separate process with its own random seed, its own opponent battles, and its own Q -table. The failure mode therefore does not apply, and our per-member solo win rates (Figure 5, left) do not underperform a single-agent baseline.

6.4 Limitations

Enumerate the honest gaps. Treat each as a real limitation, not a teaser for hypothetical future work: (1) Single opponent. All evaluations are against `SimpleHeuristicsPlayer` in `gen4ou`; generalization across opponents (e.g., `MaxBasePower` or a stronger learned agent) is untested. (2) Single 20-Pokemon pool. We do not study generalization across pools. (3) Tabular only. The inference rule itself does not require a tabular Q -function, but whether the +12.6 pp lift survives under deep value approximation is an open empirical question. (4) K capped at 10 at evaluation. This is a workstation memory limit (≈ 5.5 GiB per member; ten members concurrent), not a methodological choice. The K -saturation curve already plateaus by $K=10$, so the saturating shape itself is robust to this cap, but a tighter saturation point at higher K cannot be ruled out from these data. (5) Homogeneous per-member configuration. Every member uses the fixed winning configuration of the prior factorial study (Appendix A); we do not vary per-member hyperparameters and so cannot speak to whether the ensemble gain depends on homogeneity.

7 Conclusion

Two short paragraphs.

Paragraph 1: restate the contribution and the headline number. “This paper introduces a read-only inference rule for tabular Q -function ensembles with heuristic-prior fallback for unseen state-action pairs (Equation 4), and uses it to combine $K=10$ independently-seeded `HierSmartPlayer` members. The resulting ensemble achieves $66.8\% \pm 0.3\%$ win rate against `SimpleHeuristicsPlayer` in `gen4ou`, +12.6 percentage points over a compute-matched single-agent baseline trained on the same total simulated-battle budget.”

Paragraph 2: open directions as natural follow-ups, not promises. (i) per-member configuration robustness (does the gain hold if members are drawn from across the V5 hyperparameter grid, not just its winning cell?); (ii) diversity sources beyond seed (per-member pool bagging, alternative heuristic-init scoring functions); (iii) the K vs. evaluation-memory bound (a higher-memory evaluation could trace the saturation curve further). Explicitly do NOT claim deep-RL ensemble extensions as imminent work — those have not been implemented or tested.

References

Cameron Angliss, Jiaxun Cui, Jiaheng Hu, Arrasy Rahman, and Peter Stone. VGC-Bench: A benchmark for generalizing across diverse team strategies in competitive Pokémon, 2025.

- Oron Anschel, Nir Baram, and Nahum Shimkin. Averaged-DQN: Variance reduction and stabilization for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2017.
- Xinyue Chen, Che Wang, Zijian Zhou, and Keith Ross. Randomized ensembled double Q-learning: Learning fast without a model. In *International Conference on Learning Representations (ICLR)*, 2021.
- Stefan Faußer and Friedhelm Schwenker. Ensemble methods for reinforcement learning with function approximation. In *Multiple Classifier Systems (MCS 2011)*, volume 6713 of *Lecture Notes in Computer Science*, pages 56–65. Springer, 2011. doi: 10.1007/978-3-642-21557-5_8.
- Jake Grigsby, Yuqi Xie, Justin Sasek, Steven Zheng, and Yuke Zhu. Human-level competitive Pokémon via scalable offline reinforcement learning with transformers. In *Reinforcement Learning Conference (RLC)*, 2025.
- Sihao Hu, Tiansheng Huang, and Ling Liu. PokeLLMon: A human-parity agent for Pokémon battles with large language models, 2024.
- Akshay Kalose, Kris Kaya, and Alvin Kim. Optimal battle strategy in Pokémon using reinforcement learning. <https://web.stanford.edu/class/aa228/reports/2018/final151.pdf>, 2018. Stanford AA228 final project.
- Seth Karten, Andy Luu Nguyen, and Chi Jin. PokéChamp: An expert-level minimax language agent, 2025.
- Qingfeng Lan, Yangchen Pan, Alona Fyshe, and Martha White. Maxmin q-learning: Controlling the estimation bias of q-learning. In *International Conference on Learning Representations (ICLR)*, 2020.
- Kimin Lee, Michael Laskin, Aravind Srinivas, and Pieter Abbeel. SUNRISE: A simple unified framework for ensemble learning in deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2021.
- Scott Lee and Julian Togelius. Showdown AI Competition. In *IEEE Conference on Computational Intelligence and Games (CIG)*, pages 191–198, 2017.
- Zhixuan Lin, Pierluca D’Oro, Evgenii Nikishin, and Aaron Courville. The curse of diversity in ensemble-based exploration. In *International Conference on Learning Representations (ICLR)*, 2024.
- Ian Osband, Charles Blundell, Alexander Pritzel, and Benjamin Van Roy. Deep exploration via bootstrapped DQN. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- Oren Peer, Chen Tessler, Nadav Merlis, and Ron Meir. Ensemble bootstrapping for q-learning. In *International Conference on Machine Learning (ICML)*, 2021.
- Haris Sahovic. poke-env: A python interface for training reinforcement learning pokémon bots. <https://github.com/hsahovic/poke-env>, 2020. Open-source library; first release 2019, broadly cited as 2020 in subsequent literature.
- Diogo A. Simões, Simão Reis, Nuno Lau, and Luís P. Reis. Competitive deep reinforcement learning over a Pokémon battling simulator. In *IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, pages 40–45, 2020. doi: 10.1109/ICARSC49921.2020.9096092.
- Arzaan Singh and Zizhan Zheng. PokeAgent: “skip the tall grass” — heuristic initialization for hierarchical RL in Pokémon battles. Poster presented at Tulane School of Science and Engineering Spring 2026 Expo; full technical report in preparation, 2026.
- Arzaan Singh, Ian Kreger, and Michael Weild. Pokémon battling agents: Implementation and discussion of various RL algorithms. Technical report, Tulane University, 2025. CMPS 4740 final project.

- Yanjie Song, P. N. Suganthan, Witold Pedrycz, Junwei Ou, Yongming He, Yingwu Chen, and Yutong Wu. Ensemble reinforcement learning: A survey. *Applied Soft Computing*, 149:110975, 2023. doi: 10.1016/j.asoc.2023.110975.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- Jett Meng Wang. Winning at Pokémon random battles using reinforcement learning. M.eng. thesis, Massachusetts Institute of Technology, 2024. URL <https://dspace.mit.edu/handle/1721.1/153888>.
- Christopher J. C. H. Watkins. *Learning from Delayed Rewards*. PhD thesis, King’s College, University of Cambridge, 1989.
- Marco A. Wiering and Hado van Hasselt. Ensemble algorithms in reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 38(4):930–936, 2008. doi: 10.1109/TSMCB.2008.920231.
- Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.

A Inherited components from prior work

The per-member training pipeline used in this paper inherits two components from our earlier factorial study [Singh and Zheng, 2026]. We summarize them here for self-containedness; their design choices and the experiments that selected them are contained entirely in that earlier work.

Hierarchical action decomposition. Each member’s policy is split into a master agent that decides between attacking and switching, and a sub-agent that decides which Pokemon to switch to (conditional on the master choosing to switch). The master agent observes a 20-tuple state with $|\mathcal{A}_m| \leq 5$ actions (move 1 through 4 plus switch); the sub-agent observes a 17-tuple state with $|\mathcal{A}_s| \leq 5$ candidates. Both tables are maintained jointly via Watkins $Q(\lambda)$ on the same dense reward signal.

Heuristic Q-value initialization. Newly encountered state-action pairs are seeded with a small directional bias derived from the domain. For a move action, the score is the product of type effectiveness, same-type-attack-bonus, base power, and accuracy. For a switch action, the score is determined by type matchup, speed tier, and HP fraction of the candidate Pokemon. Scores are linearly mapped into the range $[-0.01, +0.01]$, small enough that a single observed reward overrides the prior on the first visit. We refer to the deterministic function from (s, a) to this initial value as $h(s, a)$ throughout the paper.

Why these were used as fixed inputs to V6. The prior factorial study evaluated the cross product of two architecture choices, two initialization strategies, and two exploration schedules across eight hyperparameter combinations and ten runs each (640 independent training runs, 32 million simulated battles). The single best cell in that study was *hierarchical* $\times$ *smart-init* $\times$ *fixed* ε at $\alpha = 0.1, \gamma = 0.99, \lambda = 0.7$, which reached a final rolling win rate of approximately 59.8% against SimpleHeuristicsPlayer in gen4ou. We used this single configuration unmodified for every ensemble member in this paper, treating member training as a fixed input rather than a design variable. Investigating whether the ensemble gain is robust to the choice of per-member configuration is left to future work.

B Memory bound on the inference rule

The read-only construction in Equation 4 keeps the inference-time memory footprint independent of evaluation length. A naive alternative cache the heuristic prior back into each member’s Q-table on every unseen state encountered, which causes $|Q_m^k|$ to grow linearly in the number of unique states the ensemble encounters during evaluation. In our setting (10 members, dense Watkins $Q(\lambda)$ traces, $|\mathcal{S}|$ effectively in the millions), this growth is enough that a 100,000-battle evaluation of the cached

implementation was killed by the workstation’s OOM killer roughly 30,000 battles in. The read-only formulation eliminates this failure mode by construction.

C Reproducibility details

Per-member hyperparameters. $\alpha = 0.1$, $\gamma = 0.99$, $\lambda = 0.7$, $\varepsilon = 0.05$ (fixed). Replacing eligibility traces; cleared after non-greedy actions. Reward shaping: terminal ± 1 plus $0.1\Delta_{\text{faint}} + 0.05\Delta_{\text{HP}} + 0.01\Delta_{\text{status}}$.

Pool and team builder. 20 Pokemon, all OU-legal in Gen 4, fixed across all members. Teams generated by IndexedTeambuilder: battle index n is mapped deterministically to a team via a seeded PRNG, so members and opponents encounter identical matchups at the same battle index.

Training schedule. $K_{\text{max}} = 30$ members; $\text{seed}_k = 10000 + k$; 10^6 battles each; $\approx 3 \times 10^7$ battles total.

Hardware. Eight AWS EC2 c5.4xlarge cloud instances (16 vCPU, 32 GiB each) running in parallel for fifteen days. Cloud computing costs were covered by a Tulane Newcomb-Tulane College URAF CAIDS: Focus on Data research grant.

Evaluation protocol. 100,000 battles per condition. Win rates reported with Wilson 95% confidence intervals [Wilson, 1927]. Each evaluation condition’s `eval_*.json` output, including raw win counts and timing, is distributed alongside the code at https://github.com/arzaansingh/poke_rl.

Software versions. `poke-env` 0.10; Pokemon Showdown server, Gen 4 ruleset; Python 3.12. Specific commit hashes are recorded in the `ensemble_results/run_1/params.json` output file.

D Acknowledgments and prior collaborators

This work was supported by a Tulane Newcomb-Tulane College URAF CAIDS: Focus on Data research grant (Spring 2026).

The PokeAgent research line began as a CMPS 4740 group project in Fall 2025 at Tulane University [Singh et al., 2025], jointly carried out by Arzaan Singh, **Ian Kreger**, and **Michael Weild**. We thank Ian Kreger for the original environment setup and `poke-env` integration, including the first tabular Q-learning baseline against which all subsequent agents (including the ensemble members studied in this paper) were compared. We thank Michael Weild for the LLM-assisted action reasoning study and for collaborating on the first deep Q-network implementation. That group report established the per-member tabular and hierarchical machinery on which the V5 factorial study [Singh and Zheng, 2026] and the V6 ensemble in this paper build.